

KnowledGPT: Enhancing Large Language Models with Retrieval and Storage Access on Knowledge Bases

Xintao Wang, Qianwen Yang, Yongting Qiu, Jiaqing Liang,
Qianyu He, Zhouhong Gu, Yanghua Xiao, Wei Wang

Fudan University

{xtwang21, qwyang22, 22210240256, qyhe21}@m.fudan.edu.cn,
{zhgu20, shawyh, weiwang1}@m.fudan.edu.cn, l.j.q.light@gmail.com

Abstract

Large language models (LLMs) have demonstrated impressive impact in the field of natural language processing, but they still struggle with several issues regarding, such as completeness, timeliness, faithfulness and adaptability. While recent efforts have focuses on connecting LLMs with external knowledge sources, the integration of knowledge bases (KBs) remains understudied and faces several challenges. In this paper, we introduce KnowledGPT, a comprehensive framework to bridge LLMs with various knowledge bases, facilitating both the retrieval and storage of knowledge. The retrieval process employs the program of thought prompting, which generates search language for KBs in code format with pre-defined functions for KB operations. Besides retrieval, KnowledGPT offers the capability to store knowledge in a personalized KB, catering to individual user demands. With extensive experiments, we show that by integrating LLMs with KBs, KnowledGPT properly answers a broader range of questions requiring world knowledge compared with vanilla LLMs, utilizing both knowledge existing in widely-known KBs and extracted into personalized KBs.

1 Introduction

Large Language Models (LLMs) (OpenAI, 2023) (Chiang et al., 2023) (Taori et al., 2023) have achieved substantial impact across a variety of natural language processing (NLP) tasks like translation (Wang et al., 2023), summarization (Zhang et al., 2023) and question answering (van Sonsbeek et al., 2023), alongside with all kinds of requests from real-world users. Their remarkable capabilities stem from the ever-increasing number of parameters and training data, which expands in correlation with their massive knowledge and emergent abilities like chain-of-thought reasoning (Kojima et al., 2022) and in-context learning (Brown et al., 2020).

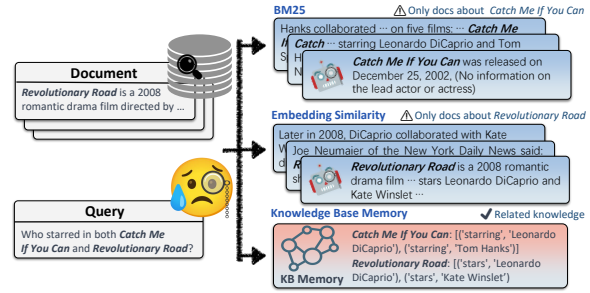


Figure 1: Comparison between retrieval results from document corpus and knowledge bases. In this case, no document retrieved from the corpus provides enough knowledge for the query, while from the knowledge base sufficient related knowledge can be retrieved.

However, LLMs still struggle with factual knowledge considering issues like completeness, timeliness, faithfulness and adaptability (OpenAI, 2023) (Kandpal et al., 2023). Firstly, LLMs demonstrate limitations in terms of timely updates and expertise in specific domains. Secondly, these models can generate unfaithful or “hallucinated” knowledge, posing both reliability and ethical concerns. Thirdly, due to constraints like costs and accessibility, LLMs can hardly incorporate new knowledge via continued training, which hampers the ability to tailor these models to accommodate specific knowledge demands. Therefore, these knowledge demands encourage comprehensive research efforts towards integrating LLMs with external sources of knowledge.

Towards this issue, some recent efforts have been made to enable LLMs to access plug-and-play knowledge sources like knowledge bases (KBs) (Modarressi et al., 2023), search engines (Schick et al., 2023), document memories (Liu, 2022), and databases (Hu et al., 2023) to provide LLMs with world knowledge, generally via LLM-generated API calls. In this paper, we focus on knowledge bases (KBs), a unique form of knowledge source featuring entity-centric knowl-

edge like relational triples and entity descriptions. On one hand, various KBs have been constructed for their practical effectiveness for applications, and the conciseness, expressiveness, interpretability and visibility of their representation. On the other hand, previous approaches have largely focused on document corpus, which reveals several deficiency when applied to KGs, as shown in Fig 1. Therefore, connecting LLMs with KBs is of significant importance, yet still remains underexplored.

Recently, several works have attempted to connect LLMs to KBs. Toolformer (Schick et al., 2023) queries Wikipedia for descriptions of interested entities to answer related questions. Graph-Toolformer (Zhang, 2023) and ToolkenGPT (Hao et al., 2023) make LLMs reason over knowledge graphs like Freebase. RET-LLM (Modarressi et al., 2023) builds personalized KG memory with relational triples extracted from past conversations for future use, in parallel with practical efforts of KG Index in LangChain (Chase, 2022) and Llama Index (Liu, 2022).

However, there are still many challenges in this direction, as shown in Figure 2. Firstly, the process by which LLMs navigate through knowledge bases for intricate and varied questions remains a problem, especially for multi-hop questions which requires information across multiple and nested entries in KBs. Secondly, aligning entities and relations in knowledge bases with their text mentions is a challenging task, as they need to map to a wide spectrum of natural language expressions and account for severe ambiguity in the knowledge bases. Thirdly, while the triple-based representation in KGs is neat and interpretable, it only covers limited information compared with natural language, which suggests the need for new representation form in KBs for LLMs.

In this paper, we propose a comprehensive framework, KnowledGPT, to connect LLMs to various knowledge bases effectively, with an improved capability in dealing with complex questions, ambiguity and knowledge representation. KnowledGPT implements a unified accessor interface for operations on different KBs, including widely-used public KBs and personalized KB memories. KnowledGPT accesses entity-oriented knowledge, including both entity descriptions and relational triples. For a given query, KnowledGPT searches KBs with three steps: search code generation, search execution, and answer generation. Inspired

by (Chen et al., 2022), KnowledGPT adopts *program of thoughts (PoT)* prompting, interacting with KBs by generating Python code which delegates searching steps and executing it. This code encapsulates functions for assessing KBs such as *entity_linking*. Afterwards, KnowledGPT integrates the retrieved knowledge to generate the response. If KnowledGPT judges that the question does not necessitate knowledge from KBs, or if the retrieved knowledge is inadequate or absent, the question will be directly answered by the LLM. Besides, KnowledGPT can also extract knowledge from unstructured texts represented in various forms to enrich the personalized KB.

Overall, our contributions are summarized as follows:

1. We propose KnowledGPT, a comprehensive framework to enable LLMs to retrieve knowledge from knowledge bases. It significantly advances the collaboration between LLMs and KBs towards vital practical challenges like complex searching and ambiguity.
2. We propose the use of personalized knowledge bases as symbolic memory for LLMs, encapsulating entity-oriented knowledge in three forms of representations. This expands the scope of knowledge in symbolic memories compared with triples-only KBs.
3. We demonstrate the efficacy of our proposed methods with experiments. The results underscore the utility and potential of using KBs as symbolic memory for LLMs.

2 Related Works

External Knowledge and Memory for LLMs Large language models (LLMs), such as GPT-4 (OpenAI, 2023) and LLaMA (Touvron et al., 2023), have demonstrated impressive performance across various applications. However, they still struggle with knowledge considering completeness, timeliness, faithfulness and adaptability. Hence, many recent efforts have been devoted to equipping LLMs with external knowledge. Internet-augmented language models (Komeili et al., 2022) (Lazaridou et al., 2022), as well as New Bing and ChatGPT “Browse with Bing” plugin, allow LLMs to access up-to-date information with search engines or web browsers. Retrieval-augmented methods like REALM (Gua et al.,

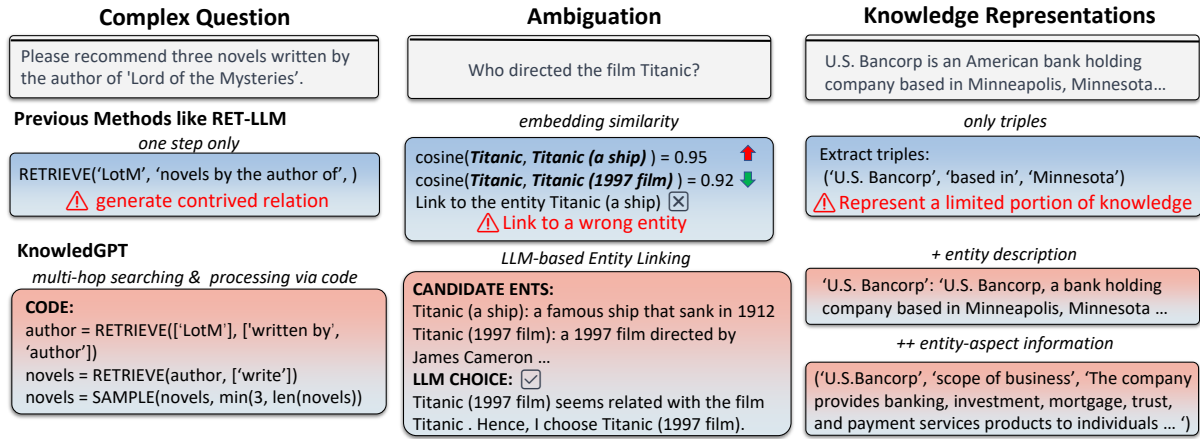


Figure 2: Comparison between KnowledGPT and previous methods towards several challenges in bridging LLMs with KBs. KnowledGPT handles complex questions through multi-hop searching and processing based on code, tackles entity ambiguation with LLM-based entity linking (middle), and provides extended forms of knowledge representations to encapsulate a broader range of knowledge from the provided text.

2020), RAG (Lewis et al., 2020), augment LLMs with document corpus, which have also been increasingly adopted by recent popular LLMs like ChatGPT as the memory unit (Liu, 2022) (Chase, 2022). ChatDB (Hu et al., 2023) augments LLMs with databases as symbolic memory.

Knowledge Bases for LLMs Some recent works have studied to augment LLMs with knowledge from external KBs or use KBs as symbolic memories, usually by making LLMs generate API calls for KB operations. Toolformer (Schick et al., 2023) trains LLMs to search Wikipedia for texts of entities. Graph-Toolformer (Zhang, 2023) empowers LLMs to reason over knowledge graphs. However, it skips the entity linking step, so it requires entity id like /m/053yx as input, instead of their names. ToolkenGPT (Hao et al., 2023) keeps the LLMs frozen and trains tool embeddings for relations in KBs to support relational queries. RET-LLM (Modarressi et al., 2023), similar to the KG memory of LangChain (Chase, 2022) and Llama-Index (Liu, 2022), extracts relational triples from user inputs and store them in a symbolic KG memory. Compared with previous efforts, KnowledGPT supports various knowledge representations and both public and private KBs, as shown in Table 1.

Knowledge-based Question Answering (KBQA) is to search for answer entities or relations given natural language queries specific to certain KGs. Existing KBQA systems are mainly based on semantic parsing (Berant et al., 2013) or information extraction (Yao and Van Durme, 2014),

where language models are increasingly involved. Semantic parsing methods (Yu et al., 2023; Cao et al., 2022; Zhang et al., 2022b; Abdelaziz et al., 2021; Lai et al., 2016) leverage semantic parser to convert natural language queries into intermediate logic forms such as SPARQL (Prud’hommeaux, 2011) and program (Liang et al., 2016), which are executed on KBs to obtain the answers. However, the generated logic forms are usually non-executable, thus failing to arrive at the correct answer (Sun et al., 2020). Pangu (Gu et al., 2022) trains a language model discriminator to evaluate probability of candidate plans. Information extraction methods usually combines retrieval and reasoning (Zhang et al., 2022a; Shi et al., 2021; Sun et al., 2019; Jiang et al., 2023; Baek et al., 2023). These methods show effectiveness in handling single-hop retrieval. However, they encounter challenges with multi-hop retrieval concerning storage and computation costs, where the number of relations expands exponentially with each added hop.

KnowledGPT differs from KBQA methods in two aspects. First, many KBQA methods are designed for special queries about relational triples in KGs, while KnowledGPT augments LLMs to respond to various user queries with knowledge in various forms from KBs. Second, KBQA methods are typically trained on specific datasets and KGs, whereas KnowledGPT requires no training and can easily accommodate different LLMs and KBs.

Method	Ent Desc	Triples	Ent Aspect Info	Multi-Hop Search	Zero-Shot	Ent Linking	External KBs	Private KB
Toolformer	✓	✗	✗	✓	✗	✗	✓	✗
ToolkenGPT	✗	✓	✗	✓	✗	✗	✓	✗
Graph-Toolformer	✗	✓	✗	✗	✗	✗	✓	✗
RET-LLM	✗	✓	✗	✗	✗	✓	✗	✓
LangChain KG Memory	✗	✓	✗	✗	✓	✓	✗	✓
Llama-Index KG Index	✗	✓	✗	✗	✓	✓	✗	✓
KnownGPT	✓	✓	✓	✓	✓	✓	✓	✓

Table 1: Comparison between KnownGPT and existing KB-augmented methods. Ent, Rel, and Desc are abbreviations for entity, relation and description, respectively.

3 Methods

In this section, we introduce KnownGPT, a comprehensive framework to integrate LLMs with KBs. We first provide the definition of two tasks of KnownGPT, knowledge retrieval and knowledge storage (Sec 3.1). Then, we elaborate the details in the retrieval (Sec 3.2) and storage (Sec 3.3) process of KnownGPT.

3.1 Task Definition

KnownGPT supplements LLMs with external knowledge from various knowledge bases (KBs), including a personalized KB (PKB) as an writable symbolic memory. Given a user input in natural language, KnownGPT undertakes two primary tasks, namely knowledge retrieval and knowledge storage. In the knowledge retrieval task, the model searches through provided KBs to retrieve relevant knowledge to answer the user query. In the knowledge storage task, the model extracts knowledge from the user input and inserts it into the PKB.

3.2 Knowledge Retrieval

KnownGPT follows a three-step process to answer user queries with knowledge from KBs, as shown in Fig 3. First, it generates a piece of search code as a logical form for query-specific KB access. Then, the search code is executed to retrieve related knowledge. Finally, KnownGPT reads the retrieved knowledge and answers the query.

We utilize the *program of thought (PoT)* prompting approach (Chen et al., 2022), which adopts Python code as the search language generated by LLMs. In this paper, we use GPT-4 (OpenAI, 2023) as the LLM. The code is encapsulated in a *search* function, as is shown in the yellow part of Fig 3, which includes built-in Python functions and three custom KB functions designed to facilitate the interaction of LLMs with KBs:

1. *get_entity_info*, which accepts an entity as input and returns its encyclopedic description.

2. *find_entity_or_value*, which accepts a query consisting of an entity and a relation as input, and outputs a list of the corresponding entity or value.

3. *find_relationship*, which accepts two entities as input, and returns a list of their relationship.

Specially, each entity or relation is represented as a list of candidate aliases, rather than a single name, to effectively handle synonyms. Besides the outputs stated above, these KB functions also return a message which logs the function call and result. Then, the overall output of the *search* function is obtained by concatenating the messages from individual KB function calls. The prompt is shown in 7.

The *search* function is then executed to retrieve the expected knowledge from KBs. The code would be decorated before execution, e.g. with a try-except statement and KB-specific accessor object, as is elaborated in Sec 3.2.1. The *search* function is executed for each KB respectively in parallel, and their results are concatenated.

Finally, the retrieved knowledge is provided to LLMs, and LLMs are tasked with responding to the user’s query, supported by the retrieved knowledge. The prompt is shown in Sec 7. LLMs will ignore the retrieved information and address the user query independently in scenarios where LLMs judge the question does not require external knowledge or the retrieved knowledge is not enough for the query.

3.2.1 Code Implementation

Next, we introduce the implementation of the KB functions to execute the generated code. We implement the functions at two levels: a unified level and a KB-specific level.

Functions at the unified level provide a unified interface for operations over different KBs. These include the three KB functions (*get_entity_info*, *find_entity_or_value*, *find_relationship*) generated

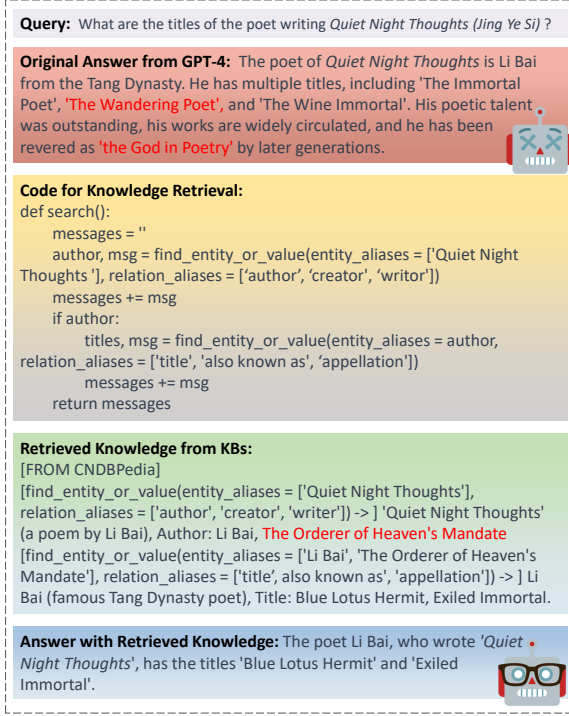


Figure 3: The detailed process of KnowledGPT’s retrieval steps for the given example. It shows the generated code for retrieval, the retrieval results, and the knowledge-enhanced response for the query, along with the original answer from LLMs. Texts highlighted in red indicates inaccuracies.

directly by LLMs, and an *entity_linking* function to align entity aliases generated by LLMs with entities in KBs.

Functions at the KB-specific level implement operations on each specific KB by calling corresponding APIs. Basically, we only need to implement three functions for each KB: *_get_entity_info*, *_entity_linking*, *_get_entity_triples*. We denote these functions with an underscore in front in this paper.

Prior to execution, we decorate the generated code. We wrap the code with a try-except statement, so that if the code breaks down in subsequent steps, the *search* function still returns valuable results from successful steps. Also, we pass the user query into the *search* function as a global variable.

3.2.2 Entity Linking

Entity linking, which aligns entity mentions in natural language with entities in a KB, is an indispensable step towards the integration of LLMs with KBs. It is essential because an entity can be referred to by various mentions (e.g. *Donald Trump* and *President Trump*), and a noun phrase can also refer to different entities (e.g. the fruit *apple* and

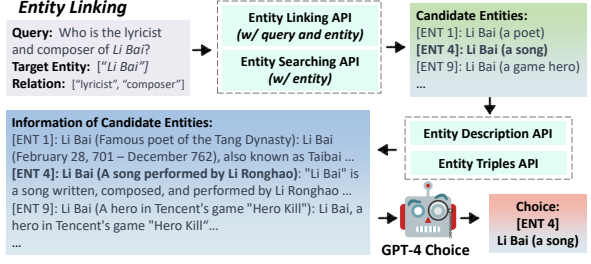


Figure 4: The detailed process of KnowledGPT’s entity linking steps for the given example.

the company *Apple*).

Our *entity_linking* function comprises three steps, as depicted in Fig 4. First, we invoke the KB-specific *_entity_linking* function to obtain candidate entities. It basically takes as input the query and entity aliases, and utilizes the entity linking API (with both entity names and the context) and searching API (with only entity names) provided by the corresponding KB. Second, we call the *_get_entity_info* function (introduced in Sec. 3.2.3) to gather information about the candidates. Each piece of entity information will be truncated to a maximum length. Finally, we provide LLMs with the function input (including the query, aliases of entity and relation) and the candidate entities along with their information, and make LLMs determine the most appropriate entity.

3.2.3 Get Entity Information

The *get_entity_info* function retrieves information about a specific entity. It first employs the *entity_linking* function to link entity aliases to an entity in the KB. Subsequently, it invokes the KB-specific *_get_entity_info* function, which returns information of the given entity in KB, including its entity description and triple information. The *_get_entity_triples* function is called to collect its triples. The KB-specific *_get_entity_info* function is nested in the *entity_linking* function, which makes it an integral part of all KB functions at the unified level.

3.2.4 Find Entity or Value

Given a query composed of an entity and a relation, the *find_entity_or_value* function is designed to retrieve the corresponding entity or attribute value. This function undergoes several steps, as is presented in Algorithm 1. Still, it starts by invoking the *entity_linking* function to associate entity aliases with a corresponding entity in the KB. Then, it calls the internal *_find_entity_or_value* function, which includes a KB-specific *_get_entity_triples*

Algorithm 1: find_entity_or_value

Input: query q , alias list of entity $\mathcal{E}$, alias list of relation $\mathcal{R}$.
Output: a list of target entities or attribute values $\mathcal{T}$.

```
1 Function EMBSIM( $str\ r, list[str]\ \mathcal{R}$ ):  
2    $s = -1$   
3    $\mathbf{r} = \text{embedding}(r)$   
4   for  $r_i \in \mathcal{R}$  do  
5      $\mathbf{r}_i = \text{embedding}(r_i)$   
6     if  $\cos(\mathbf{r}, \mathbf{r}_i) > s$  then  
7        $s = \cos(\mathbf{r}, \mathbf{r}_i)$   
8   return  $s$   
9  $e = \text{entity\_linking}(\mathcal{E})$   
10 if  $e == \text{NULL}$  then  
11   return  $\text{NULL}$   
12  $r = \text{NULL}$   
13  $s_r = -1$   
14 for  $\text{triple} \in \text{triples}$  do  
15    $r_i = \text{triple.rel}$   
16    $s_i = \text{EMBSIM}(r_i, \mathcal{R})$   
17   if  $s_i > s_r$  then  
18      $s_r = s_i$   
19      $r = r_i$   
20 if  $r == \text{NULL}$  then  
21   return  $\text{NULL}$   
22  $\text{triples}_r = \text{triples}$  with relation  $r$   
23  $\mathcal{R} = \text{target entities or attribute values in } \text{triples}_r$   
24 return  $\mathcal{R}$ 
```

functions that retrieve all triples related to the entity. Subsequently, the relations in these triples are sorted based on their similarity with the input relation aliases. Here we employ cosine similarity of sentence embeddings, rather than symbolic metrics, which considers synonyms of relations. Afterwards, we select the relation with highest similarity score, and return entities or attribute values from all corresponding triples. To improve the robustness of our method, we will conduct a further search within the entity description for the relation if no triples are found. If the relation is present in the description, we return the corresponding sentence. Otherwise, we return the whole description, which may still offer relevant details for LLMs.

3.2.5 Find Relationship

Given a query composed of two entities, the *find_relationship* function is designed to retrieve their relationship. This function is similar to *find_entity_or_value*. The difference is that, upon retrieving triples or entity information for the first entity, the *find_relationship* function proceeds to search for the second entity, instead of the relation. If this initial search fails, the function swaps the first entity and second entity and searches again. Different from relation similarity, we measure entity similar-

ity by Levenshtein distance d . The entity similarity is calculated as $100 - d$ if two entity names have word overlap, and 0 otherwise.

3.3 Knowledge Storage

While public KBs provide abundant world knowledge, they are still unable to cover all knowledge that users are interested in. To meet users' personal knowledge demands, KnowledGPT introduces a personalized KB (PKB) that acts as LLMs' symbolic memory, granting users the capability to store and access specialized knowledge. The PKB is populated by knowledge extracted from user-provided documents. When users intend to add knowledge into the PKB we prompt LLMs to extract knowledge from their provided documents, with the prompt shown in Sec A.

We consider knowledge represented in three forms, including entity description, relational triples, and entity-aspect information, as is shown in Fig 2, which is different from RET-LLM (Modarressi et al., 2023) or the KG-Index of LangChain (Chase, 2022) and Llama Index (Liu, 2022) that extract only triples. While entity description and relational triples have been widely adopted in knowledge bases like Wikipedia and Wikidata, they only represent a limited portion of knowledge, as is shown by experiments in Sec 4.4. For example, when we want to know the experience of Socrates as a soldier, most content in Socrates' wikipedia page would be hardly helpful, and it can also hardly be represented as a triple. Therefore, we propose an additional knowledge representation, termed as entity-aspect information, for symbolic memory of LLMs. It is a variation of triple where the object is a long piece of text which describes and can be retrieved by an entity and an aspect. For instance, a record might be indexed by ("Socrates", "Military Service") and correspond to the description "Socrates served as a Greek hoplite or heavy infantryman...". Knowledge represented in this form will also be retrieved also by the *get_entity_or_value* function.

Given the tiny scale of PKBs in comparison to public KBs, we consider a different strategy for entity linking on PKBs. The difference is mainly three-fold. First, we define the entity searching API for PKB based on exact match and embedding similarity. Embedding similarity aids in recognizing widely-known entity aliases, such as *Chanelle Scott Calica* and *Shystie*. Second, during extraction,

an extracted entity mention is not aligned to entities existing in the PKB. Therefore, an entity may be extracted as distinct mentions in different documents. Hence, for entity linking, KnowledGPT returns multiple matched entities. Third, an entity would be extracted with an aliases list, which would be provided to LLMs for entity linking.

For the *get_entity_or_value* function, since a relation can also be extracted as different expressions, we opt to retrieve relations with similarity score over a threshold, instead of a top-scored relation.

4 Experiments

In this section, we experiment KnowledGPT on various settings, including manually crafted diverse queries on popular KBs (Sec 4.2), knowledge-based question answering (Sec 4.3), and personalize KBs as memory for LLMs (Sec 4.4).

4.1 Experimental Setups

Knowledge Bases. KnowledGPT can access various KBs with its unified interface for KB operation. In this paper, we primarily consider the following KBs:

1. Wikipedia ¹ and Wikidata ². Wikipedia provides rich encyclopedic information about world entities, maintained by global volunteers. Wikidata is a knowledge graph complementing Wikipedia, which structures and organizes this encyclopedic knowledge in relational triples.
2. CN-DBPedia (Xu et al., 2017) is a large-scale, constantly-updated Chinese KB extracted from various sources including Chinese Wikipedia ³ and Baidu Baike ⁴. CN-DBPedia contains both entity descriptions like Wikipedia and relational triples like Wikidata.
3. Personalized KB is designed as a writable symbolic memory for LLMs. It stores upcoming knowledge extracted from user inputs.
4. The NLPCC 2016 KBQA Knowledge Base (Duan, 2016), which is widely adopted to evaluate models in terms of knowledge-based question answering tasks. It contains 43 million triples. This KB is only used in Sec 4.3.

¹<https://en.wikipedia.org/>

²<https://wikidata.org/>

³<https://zh.wikipedia.org/>

⁴<https://baike.baidu.com/>

In practical scenarios, we initiate the process by determining the language of the user query. For English queries, Wikipedia and Wikidata, as well as the personalized KB are employed. For Chinese queries, we utilize CN-DBPedia in conjunction with the personalized KB. Our methods can also be easily extended to more languages with KBs in corresponding languages.

Language Models. In this paper, we employ the powerful LLM GPT-4 by default, which is accessed by the OpenAI API ⁵. We prompt LLMs with instructions, requirements, and in-context examples, and require LLMs to output in json format. The detailed prompts are shown in Sec A. For sentence embeddings, we employ GPT text-embedding-ada-002 model from OpenAI.

4.2 Queries on Popular KBs

LLM	Direct Answer	Code	Entity Linking	KnowledGPT Answer
GPT-4	4/11	9/11	22/22	11/11
ChatGPT	4/11	8/11	13/19	4/11

Table 2: Number of successful answers on the selected samples. The results are evaluated by human annotators. For code generation, it is considered successful if the code is supposed to be able to retrieve all the necessary knowledge related.

To evaluate KnowledGPT in terms of diversified queries of real users requiring external knowledge, we craft 11 questions about knowledge from CN-DBPedia ⁶. These questions span a variety of types, including single-hop and multi-hop relational queries, relation prediction, diversified instructions, mixed queries, and value comparison. These questions concern both popular entities and long-tail entities. The detailed examples are shown in Sec B. We experiment on KnowledGPT with GPT-4 and ChatGPT as the base LLMs, and measure the rate of successful generation in terms of direct answers without KBs, code generation, entity linking, and KnowledGPT answers.

The results are shown in Table 2. For detailed generations, please refer to Sec B. We observe that: (1) GPT-4 and ChatGPT themselves are proficient at addressing queries about well-known entities, but they also hallucinate frequently about the unpopular entities. (2) KnowledGPT with GPT-4 excellently accomplish tasks like code generation and

⁵<https://platform.openai.com/>

⁶As CN-DBPedia is a Chinese KB, the queries and generations are also in Chinese. We translate them into English to ease understanding.

entity linking, and eventually answers user queries with correct knowledge, representing a marked improvement over the vanilla responses from GPT-4. (3) However, for ChatGPT, the success rate of intermediate steps remains to be improved, which constrains the overall efficacy of KnowledGPT. In the code generation step, ChatGPT sometimes generates poor relation aliases, such as *who is the father*, especially for diverse or intricate queries. This comparison suggests that GPT-4 significantly outperforms ChatGPT in aspects like complex instruction understanding, task decomposition and code generation. While we also experimented with smaller open-source LLMs like Llama-2-Chat-13B, they struggle to provide accurate answers directly and also fail to generate well-formatted code and respond in the JSON format as is required by the KnowledGPT framework.

Case Studies. Fig 3 illustrates a complete process of employing KnowledGPT to answer the question “What are the titles of the poet who wrote *Quiet Night Thoughts (Jing Ye Si)*?”, which requires multi-hop knowledge retrieval and reasoning. The original answer from GPT-4 contains untruthful information, which shows the need to augment LLMs with precise external knowledge. KnowledGPT generates an excellent piece of code, which first looks for the author of the poem *Quiet Night Thoughts*, and then searches for the author’s titles. This demonstrates the effectiveness of solving multi-hop knowledge retrieval with code. Upon execution, the code efficiently retrieves relevant information. Finally, KnowledGPT integrates the retrieved knowledge to answer the query correctly. Despite potential noise in the intermediate steps, KnowledGPT well filters the noise and answers properly.

Fig 5 illustrates three instances of the entity linking step in KnowledGPT. The examples clearly indicate that the original candidate entities, returned from the entity linking and searching api of external KBs, are not well-ordered and might not even include the correct entity. Hence, simply selecting the top entity could introduce severe noise. We apply GPT-4 to select from the candidate entities provided with their information. The results indicate that GPT-4 proficiently identifies the correct entity for the query, and is also capable of rejecting all options when none is fit.

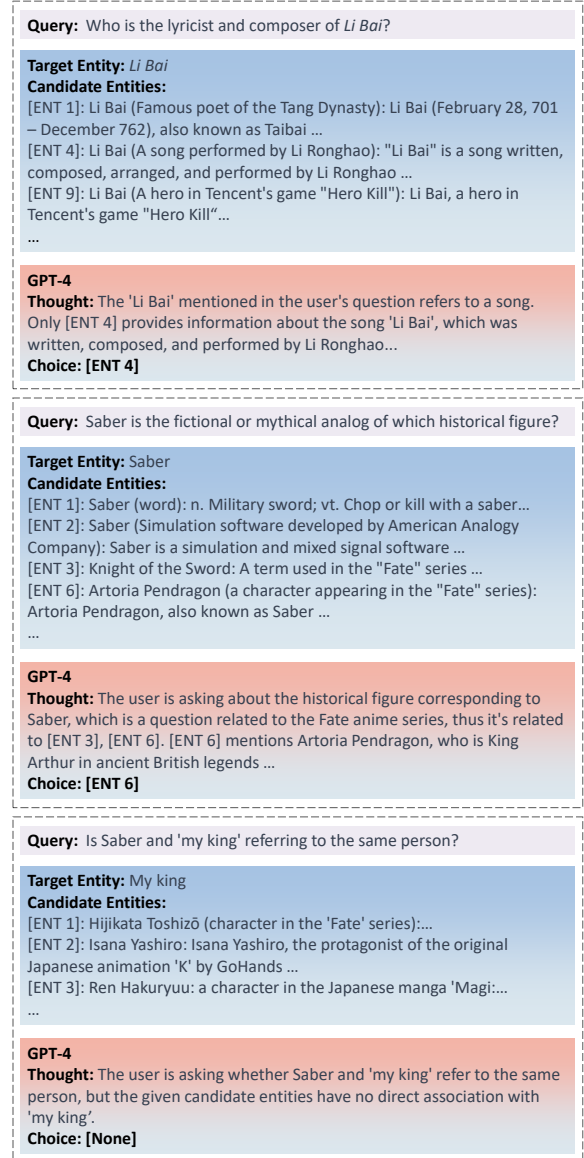


Figure 5: Case studies of KnowledGPT’s entity linking process.

4.3 Knowledge-Based Question Answering

We evaluate KnowledGPT on zero-shot knowledge-based question answering (KBQA). KBQA is an extensively researched domain that seeks to answer natural language questions specific to certain relational triples in KBs. For example, to answer the question “Who is the author of *the Republic*”, a KBQA model is expected to retrieve the triple (*the Republic*, *written by*, *Plato*) and answer *Plato*.

Given the expenses of invoking OpenAI APIs, we have compiled two compact datasets, namely NLPCC-100 for single-hop queries and NLPCC-MH-59 for multi-hop queries. NLPCC-100 is com-

posed of 100 samples from the test set of NLPCC 2016 KBQA dataset (Duan, 2016), while NLPCC-MH-59 consists of 59 samples from the test set of NLPCC-MH (Wang and Zhang, 2019), a multi-hop KBQA dataset. NLPCC-MH is automatically constructed by expanding the NLPCC 2016 dataset, which leads to certain inherent noise. From the NLPCC-MH dataset, we manually select 59 samples, ensuring their quality and the presence of supporting fact chains in the KB. Furthermore, we rectify any noise present in these samples. For both NLPCC-100 and NLPCC-MH-59, we use exclusively the full NLPCC 2016 KBQA Knowledge Base in this experiment.

We make several modification to KnowledGPT towards this dataset and KB. First, in the provided KB, the tail entities of triples may contain multiple entities separated by special symbols, so we adjust the prompt for search code generation to request LLMs to include a splitting mechanism in the generated code. Second, for better entity linking, as the provided KB does not contain entity description, we modify the implementation of *_get_entity_information* to return 10 triples related to the entity, sorted by the jaccard similarity between the query relation and the relation in triples. Third, we also modify the entity linking prompt, requiring LLMs to also adjust the query relation alias to better align with relations in the KB.

We compare KnowledGPT with the following baseline methods :

1. Retrieval via embedding similarity. Each triple is treated as a document and embedded using the CoSENT model (Ming, 2022). A single document is retrieved based on embedding similarity for each search. For multi-hop questions, the result from the first retrieval is added to the query to facilitate the second retrieval.
2. Retrieval via BM25 (Robertson et al., 1995). For each entity, we group all its triples as a document. The most relevant document is retrieved using the BM25 algorithm for each search query, with stop words removed. We regard it as successful if the retrieved document contains the corresponding triple. For multi-hop queries, we pick a triple from the initial retrieval’s document based on the jaccard similarity between relations, and integrate this triple into the subsequent retrieval’s query.

3. SPE (Lai et al., 2016). SPE extracts subject-predicate pairs from simple questions using embedding similarity. It obtained the first place in the contest of NLPCC 2016 KBQA task.

We report averaged F1 on NLPCC-100 and NLPCC-MH-59. Averaged F1 is a widely adopted metric for KBQA, designed for tasks with multiple golden answers and predictions. However, since in our dataset there is only one answer and one prediction for each sample, so the averaged F1 actually is equivalent to accuracy.

The results are shown in Table 3, from which we have the following observation: (1) For single-hop queries, KnowledGPT significantly outperforms retrieval methods via BM25 and embedding similarity, which shows the effectiveness of retrieval from symbolic KBs compared with document corpus, for questions related to knowledge in KBs. (2) Zero-shot KnowledGPT outperforms the SPE method trained on the full training set of NLPCC-2016 KBQA dataset (0.92 vs 0.85), which shows the strong zero-shot performance of KnowledGPT. (3) For multi-hop queries, KnowledGPT also achieves excellent performance, while the performance of retrieval methods based on BM25 and embedding similarity drops significantly.

Dataset	BM25	Embedding Similarity	SPE	KnowledGPT
NLPCC-100	0.71	0.31	0.85	0.92
NLPCC-MH-59	0.44	0.19	-	0.93

Table 3: Averaged F1 of different methods on NLPCC-100 and NLPCC-MH-59.

4.4 KB as Memory

We conduct experiments to study the effectiveness of KnowledGPT when paired with a modifiable personalized KB. KnowledGPT is tasked with building the PKB by extracting knowledge from provided documents, and study whether KnowledGPT answer corresponding questions properly with the PKB. Fig 6 shows an example where text about Socrates is provided. KnowledGPT extracts knowledge from the text, and answers related questions by retrieving the extracted knowledge.

We further apply KnowledGPT to HotpotQA (Yang et al., 2018), a multi-hop question answering dataset with provided documents. We select 25 questions from HotpotQA dev set (distractor), including 5 comparison questions like “Which is a shrub, Mimosa or Cryptocoryne?” and 20 bridge questions like “When was Erik Watts’ father

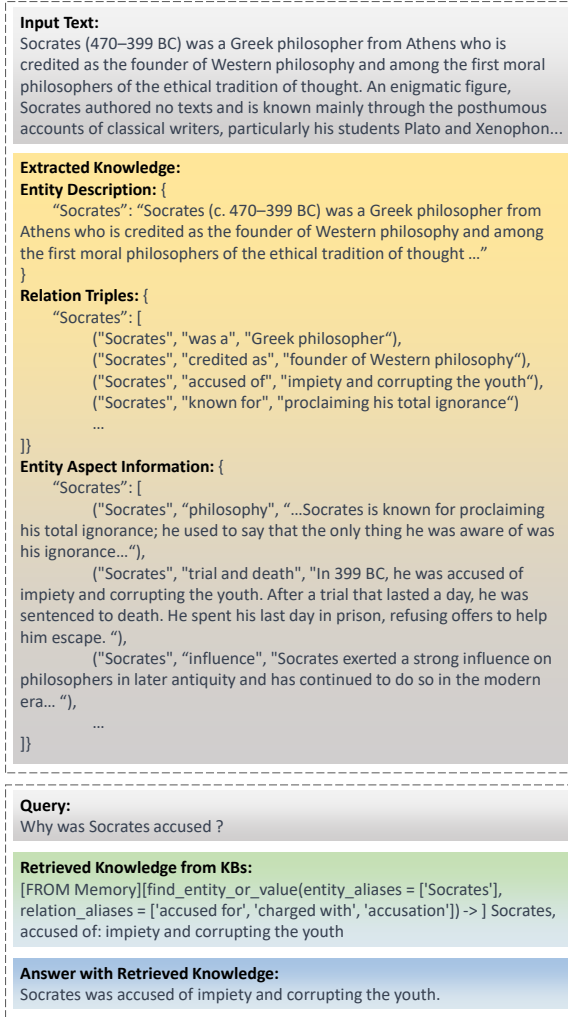


Figure 6: An example showing knowledge extraction and retrieval of KnowledGPT on the personalized KB.

born?”. Each question is paired with 10 documents, and KnowledGPT extracts knowledge from these documents to build the PKB.

The results are shown in Table 4. KnowledGPT successfully answer all comparison questions, and 15 out of 20 bridge questions. Among the failed cases, two bridge questions failed to extract the knowledge needed to answer the question and one bridge question failed in the entity linking stage. Overall, this experiment suggests the promising future of utilizing PKBs as symbolic memory of LLMs.

	Comparison	Bridge	All
KnowledGPT	5/5	15/20	20/25

Table 4: Number of successful answers on 25 questions selected from HotPotQA. The results are evaluated by human annotators.

We further investigate the knowledge extraction coverage of KnowledGPT on 100 documents from HotpotQA (Yang et al., 2018), considering various LLMs and diverse combination of applied knowledge representations. To quantify the coverage, we employ the word recall rate, calculated as

$$\frac{|W_{extracted} \cap W_{doc}|}{|W_{doc}|}, \quad (1)$$

where $|\cdot|$ indicates the cardinality of a set. $W_{extracted}$ and W_{doc} denote the set of words in the extracted knowledge and the document respectively, after preprocessing including the removal of stop words and lemmatization, utilizing the NLTK toolkit (Bird and Klein, 2009).

The results are shown in Table 5, from which we have the following observations: (1) When restricting knowledge representation solely to triples, the extraction coverage stands at 0.53, which indicates that only a limited portion of knowledge can be represented as triples. Therefore, a PKB supporting triples alone falls short of adequately encompassing the knowledge provided by real users. (2) With additional knowledge representations, i.e., entity description and entity-aspect information, we observe a marked improvement in knowledge extraction coverage, suggesting that incorporating entity description and entity-aspect information enables KnowledGPT to populate the PKB with a broader spectrum of knowledge. (3) ChatGPT and GPT-4 achieve similar proficiency for knowledge extraction. GPT-4 outperforms ChatGPT only when entity-aspect info is included, which probably is attributed to GPT-4’s enhanced capability at following complex instructions.

	w/ triples only	+ entity desc	++ entity aspect info
ChatGPT	0.53	0.66	0.81
GPT-4	0.53	0.62	0.86

Table 5: Knowledge extraction coverage of KnowledGPT on 100 documents from HotpotQA with different LLMs and various combination of applied knowledge representations.

5 Limitations

While KnowledGPT enables LLMs to effectively perform KB operations on external knowledge bases, there remain several limitations in its current form. First, the retrieval process entails a single-round of code generation and execution for efficiency concerns. However, a multi-round mecha-

nism may better allow LLMs to autonomously explore KBs. As LLMs are not aware of the contents within KBs, they might generate search that appear logical but yield no results. For example, a query like “Who is the voice actor for the heroine in ...” may require a two-hop searching for the relations *heroine* and *voice actor* subsequently in certain KBs, or just a single relation *main voice actor* in others. In these scenarios, a multi-round mechanism empowers LLMs to probe and revisit the KBs autonomously, which might yield better results but with increased costs. Second, we experiment with KnowledGPT on representative yet small datasets, constrained by the expenses of accessing GPT-4 via API. While the results validate the effectiveness of KnowledGPT, more comprehensive evaluations on full benchmarks are expected to better compare KnowledGPT to related methods. We plan to study fine-tuning LLMs like Llama for KnowledGPT in our future work to improve the efficiency and conduct more thorough experiments. Finally, it remains a practical issue when LLMs need to access external KGs, rather than solving problems independently. In this work, we simply let LLMs make this decision, while better approaches remain to be explored.

6 Conclusion

In this paper, we introduce KnowledGPT, a comprehensive framework to integrate LLMs with external knowledge bases, facilitating LLMs’ retrieval and storage on KBs. For retrieval, KnowledGPT adopts “program of thought” prompting, which retrieves knowledge via code generation and execution. For storage, KnowledGPT extracts various forms of knowledge from user provided texts, and populate the personalized KB with the extracted knowledge. KnowledGPT tackles several challenges inherent in integrating LLMs with KBs, including complex question answering, ambiguity in entity linking, and limited forms of knowledge representations. We show with extensive experiments that KnowledGPT effectively provides LLMs with the capability to operate on external KBs.

References

Ibrahim Abdelaziz, Srinivas Ravishankar, Pavan Kapanipathi, Salim Roukos, and Alexander Gray. 2021. [A semantic parsing and reasoning-based approach to knowledge base question answering](#). *Proceedings*

of the AAAI Conference on Artificial Intelligence, 35(18):15985–15987.

Jinheon Baek, Alham Fikri Aji, and Amir Saffari. 2023. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. *arXiv preprint arXiv:2306.04136*.

Jonathan Berant, Andrew K. Chou, Roy Frostig, and Percy Liang. 2013. Semantic parsing on freebase from question-answer pairs. In *Conference on Empirical Methods in Natural Language Processing*.

Edward Loper Bird, Steven and Ewan Klein. 2009. Natural language processing with python.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. [Language models are few-shot learners](#). *CoRR*, abs/2005.14165.

Shulin Cao, Jiaxin Shi, Zijun Yao, Xin Lv, Jifan Yu, Lei Hou, Juanzi Li, Zhiyuan Liu, and Jinghui Xiao. 2022. [Program transfer for answering complex questions over knowledge bases](#).

Harrison Chase. 2022. [LangChain](#).

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. [Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality](#).

Nan Duan. 2016. Overview of the nlpcc-iccpol 2016 shared task: Open domain chinese question answering. In *Natural Language Understanding and Intelligent Applications*, pages 942–948. Springer International Publishing.

Yu Gu, Xiang Deng, and Yu Su. 2022. Don’t generate, discriminate: A proposal for grounding language models to real-world environments. *arXiv preprint arXiv:2212.09736*.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR.

- Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2023. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *arXiv preprint arXiv:2305.11554*.
- Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. 2023. Chatdb: Augmenting llms with databases as their symbolic memory. *arXiv preprint arXiv:2306.03901*.
- Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. 2023. [Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph](#).
- Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. 2023. Large language models struggle to learn long-tail knowledge. In *International Conference on Machine Learning*, pages 15696–15707. PMLR.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213.
- Mojtaba Komeili, Kurt Shuster, and Jason Weston. 2022. [Internet-augmented dialogue generation](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8460–8478, Dublin, Ireland. Association for Computational Linguistics.
- Yuxuan Lai, Yang Lin, Jiahao Chen, Yansong Feng, and Dongyan Zhao. 2016. Open domain question answering system based on knowledge base. In *Natural Language Understanding and Intelligent Applications: 5th CCF Conference on Natural Language Processing and Chinese Computing, NLPCC 2016, and 24th International Conference on Computer Processing of Oriental Languages, ICCPOL 2016, Kunming, China, December 2–6, 2016, Proceedings 24*, pages 722–733. Springer.
- Angeliki Lazaridou, Elena Gribovskaya, Wojciech Stokowiec, and Nikolai Grigorev. 2022. Internet-augmented language models through few-shot prompting for open-domain question answering. *arXiv preprint arXiv:2203.05115*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.
- Chen Liang, Jonathan Berant, Quoc Le, Kenneth D Forbus, and Ni Lao. 2016. Neural symbolic machines: Learning semantic parsers on freebase with weak supervision. *arXiv preprint arXiv:1611.00020*.
- Jerry Liu. 2022. [LlamaIndex](#).
- Xu Ming. 2022. [text2vec: A tool for text to vector](#).
- Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. 2023. Ret-llm: Towards a general read-write memory for large language models. *arXiv preprint arXiv:2305.14322*.
- OpenAI. 2023. [Gpt-4 technical report](#).
- Eric Prud’hommeaux. 2011. [Sparql query language for rdf](#).
- Stephen Robertson, S. Walker, S. Jones, M. M. Hancock-Beaulieu, and M. Gatford. 1995. [Okapi at trec-3](#). In *Overview of the Third Text REtrieval Conference (TREC-3)*, pages 109–126. Gaithersburg, MD: NIST.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.
- Jiaxin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang. 2021. [Transfernet: An effective and transparent framework for multi-hop question answering over relation graph](#).
- Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. [Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text](#).
- Yawei Sun, Lingling Zhang, Gong Cheng, and Yuzhong Qu. 2020. [Sparqa: Skeleton-based semantic parsing for complex questions over knowledge bases](#).
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html>, 3(6):7.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. [Llama: Open and efficient foundation language models](#).
- Tom van Sonsbeek, Mohammad Mahdi Derakhshani, Ivona Najdenkoska, Cees GM Snoek, and Marcel Worring. 2023. Open-ended medical visual question answering through prefix tuning of language models. *arXiv preprint arXiv:2303.05977*.
- Longyue Wang, Chenyang Lyu, Tianbo Ji, Zhirui Zhang, Dian Yu, Shuming Shi, and Zhaopeng Tu. 2023. Document-level machine translation with large language models. *arXiv preprint arXiv:2304.02210*.
- Yue Wang and Richong Zhang. 2019. A dynamic programming-based approach to knowledge-based question answering. *Journal of Zhengzhou University (Science Edition)*, 51(4):37–42.

Bo Xu, Yong Xu, Jiaqing Liang, Chenhao Xie, Bin Liang, Wanyun Cui, and Yanghua Xiao. 2017. Cndbpedia: A never-ending chinese knowledge extraction system. In *International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems*, pages 428–438. Springer.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. [HotpotQA: A dataset for diverse, explainable multi-hop question answering](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.

Xuchen Yao and Benjamin Van Durme. 2014. Information extraction over structured data: Question answering with freebase. In *Proceedings of the 52nd annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 956–966.

Donghan Yu, Sheng Zhang, Patrick Ng, Henghui Zhu, Alexander Hanbo Li, Jun Wang, Yiqun Hu, William Wang, Zhiguo Wang, and Bing Xiang. 2023. [Decaf: Joint decoding of answers and logical forms for question answering over knowledge bases](#).

Jiawei Zhang. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.

Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. 2022a. [Subgraph retrieval enhanced model for multi-hop knowledge base question answering](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Minhao Zhang, Ruoyu Zhang, Yanzeng Li, and Lei Zou. 2022b. [Crake: Causal-enhanced table-filler for question answering over large scale knowledge base](#).

Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen McKeown, and Tatsunori B Hashimoto. 2023. Benchmarking large language models for news summarization. *arXiv preprint arXiv:2301.13848*.

A Prompts

The prompts are shown in Fig 7, Fig 8, Fig 9, Fig 10. These prompts primarily consist of task introductions, requirements, the format of input and output, and in-context examples. We request LLMs to output in JSON format, and in the same language as the query. The prompts emphasize the LLMs should finish special tasks, instead of directly answering the input questions. They are mostly written in English, with some in-context examples in Chinese to better support queries in

Chinese. To ease understanding, these examples are translated into English in these figures.

Prompt for search code generation first asks LLMs to judge whether the query need knowledge from external KBs. If so, it continues to generate the search code. It pre-defines three functions that LLMs can generate to access KBs. It asks LLMs to generate a list of entity aliases or relation synonyms as input to these functions.

Prompt for entity linking directs LLMs to select from multiple candidate entities for the input query and target entity, provided with information of the candidate entities. While LLMs have the option to output [NONE] if none seems related, they are advised to use this option cautiously.

Prompt for question answering first guides LLMs to assess if the provided knowledge adequately supports answering this question. If so, it asks LLMs to answer the query with the retrieved knowledge. Otherwise, we let LLMs answer this question independently.

Prompt for knowledge extraction prompts LLMs to extract knowledge from the provided text. For long texts, LLMs tend to overlook many pieces of information during the extraction process. We find that emphasizing “Do not miss any knowledge points.” largely improves the knowledge extraction coverage. Additionally, the prompt encourages LLMs to present knowledge in the form of relational triples and entity-aspect information when possible, instead of entity description.

B Detailed Results of Queries on Popular KBs

In this section, we elaborate the detailed results of the manually crafted examples mentioned in Sec 4.2. The examples are mainly in Chinese, including their queries, retrieval results, and answers. We translate them into English to ease understanding. We color incorrectly generated content that in red, including false facts or illogical code. We also color unhelpful answers in brown, where GPT-4 and ChatGPT admit their ignorance. In the two cases about value comparison, GPT-4 attempts to conduct value comparison in the code, which seems logical. However, it actually performs string comparison, which makes the results unreliable without improved value comparing functions that can take into account units and unit conversion.

Prompt for Search Code Generation:

You are an awesome knowledge graph accessing agent that helps to RETRIEVE related knowledge about user queries via writing python codes to access external knowledge sources. Your python codes should implement a search function using exclusively built-in python functions and the provided functions listed below.

===PROVIDED FUNCTIONS===

1. `get_entity_info`: obtain encyclopedic information about an entity from external sources, which is used to answer general queries like "Who is Steve Jobs". Args: "entity_aliases": a list of the entity's aliases, e.g. ['American', 'United States', 'U.S.'] for the entity 'American'. Return: two strings, 'result' and 'message'. 'result' is the encyclopedic information about the entity if retrieved, None otherwise. 'message' states this function call and its result.
2. `find_entity_or_value`: access knowledge graphs to answer factual queries like "Who is the founder of Microsoft?". Args: "entity_aliases": a list of the entity's aliases, "relation_aliases": a list of the relation's aliases. Return: two variables, 'result' and 'message'. 'result' is a list of entity names or attribute value to this query if retrieved, None otherwise. 'message' is a string states this function call and its result.
3. `find_relationship`: access knowledge graphs to predict the relationship between two entities, where the input query is like "What's the relationship between Steve Jobs and Apple Inc?". Args: "entity1_aliases": a list of entity1's aliases, "entity2_aliases": a list of entity2's aliases. Return: two strings, 'result' and 'message'. 'result' is the relationship between entity1 and entity2 if retrieved, None otherwise. 'message' states this function call and its result.

===REQUIREMENTS===

1. [IMPORTANT] Always remember that your task is to retrieve related knowledge instead of answering the queries directly. Never try to directly answer user input in any form. Do not include your answer in your generated 'thought' and 'code'.
2. Exclusively use built-in python functions and the provided functions.
3. To better retrieve the intended knowledge, you should make necessary paraphrase and list several candidate aliases for entities and relations when calling the provided functions, sorted by the frequency of the alias. E.g., "Where is Donald Trump born" should be paraphrased as `find_entity_or_value(["Donald Trump", "President Trump"], ["place of birth", "is born in"])`. Avoid entity alias that may refer to other entities, such as 'Trump' for 'Donald Trump'.
4. When using `find_entity_or_value`, make sure the relation is a clear relation. Avoid vague and broad relation aliases like "information". Otherwise, use `get_entity_info` instead. For example, for the question "Who is related to the Battle of Waterloo?", you should use `get_entity_info(entity_aliases = ['the Battle of Waterloo'])` instead of `find_entity_or_value(entity_aliases = ['the Battle of Waterloo'], relation_aliases = ['related to'])` since 'related to' is too vague to be searched.
5. The input can be in both English and Chinese. If the input language is NOT English, make sure the args of `get_entity_info`, `find_entity_or_value` and `find_relationship` is in the input language.
6. The queries may need multiple or nested searching. Use smart python codes to deal with them. Note that `find_entity_or_value` will return a list of results.
7. Think step by step. Firstly, you should determine whether the user input is a query that "need knowledge". If no, simply generate "no" and stop. Otherwise, generate "yes", and go through the following steps: First, Come up with a "thought" about how to find the knowledge related to the query step by step. Make sure your "thought" covers all the entities mentioned in the input. Then, implement your "thought" into "code", which is a python function with return. After that, make an "introspection" whether your "code" is problematic, including whether it can solve the query, can be executed, and whether it contradicts the requirements (especially whether it sticks to the RETRIEVE task or mistakenly tries to answer the question). Make sure "thought" and "introspection" are also in the same language as the query. Finally, set "ok" as "yes" if no problem exists, and "no" if your "introspection" shows there is any problem.
8. For every call of `get_entity_info`, `find_entity_or_value` and `find_relationship`, the return 'message' are recorded into a string named 'messages', which is the return value of `search()`.
9. Add necessary explanation to the 'messages' variable after running certain built-in python codes, such as, `messages += f'{top_teacher}'` is the teacher with most citations'.
10. When the user query contains constraints like "first", "highest" or mathematical operations like "average", "sum", handle them with built-in functions.
11. Response in json format.

===OUTPUT FORMAT===

```
{
  "need_knowledge": "<yes or no. If no, stop generating the following.>"
  "thought": "<Your thought here. Think how to find the answer to the query step by step. List possible aliases of entities and relations.>",
  "code": "def search():\n\tmessages = \"\n\t<Your code here. Implement your thought.>\n\treturn messages\n",
  "introspection": "<Your introspection here.>",
  "ok": "<yes or no>"
}
```

===EXAMPLES===

1.
Input: "Who are you?"
Output: {

```
  "need_knowledge": "no"
}
```

2.
Input: "Who proposed the theory of evolution?"
Output: {

```
  "need_knowledge": "yes",
  "thought": "The question is asking who proposed the theory of evolution. I need to search for the proponent of the theory of evolution. The possible expressions for the 'proponent' relationship include 'proposed', 'proponent', and 'discovered'.",
  "code": "def search():\n\tmessages = \"\n\t\tproposer, msg = find_entity_or_value(entity_aliases = ['theory of evolution'], relation_aliases = ['propose', 'proponent', 'discover'])\n\t\tmessages += msg\n\treturn messages\n",
  "introspection": "The generated code meets the requirements.",
  "ok": "yes"
}
```

3.
Input: "what is one of the stars of 'The Newcomers' known for?"
Output: {

```
  "need_knowledge": "yes",
  "thought": "To answer this question, firstly we need to find the stars of 'The Newcomers'. The relation can be paraphrased as 'star in', 'act in' or 'cast in'. Then, we should select one of them. Finally, we should retrieve its encyclopedic information to know what he or she is known for. We should not treat 'known for' as a relation because its too vague.",
  "code": "def search():\n\tmessages = \"\n\t\tstars, msg = find_entity_or_value(entity_aliases = ['The Newcomers'], relation_aliases = ['star in', 'act in', 'cast in'])\n\t\tmessages += msg\n\t\tstar = random.choice(stars)\n\t\tstar_info, msg = get_entity_info(entity_aliases = [star])\n\t\tmessages += msg\n\treturn messages\n",
  "introspection": "The generated code is executable and matches user input. It adheres to the requirements. It finishes the retrieve task instead of answering the question directly.",
  "ok": "yes"
}
```

Figure 7: Prompt for search code generation. The second example is in Chinese and translated into English.

Prompt for Entity Linking:

You are an awesome knowledge graph accessing agent. There are many entities with similar names that exist in knowledge graphs which cause ambiguity, such as the fruit 'apple' and the company 'Apple'. Given the user input, and the interested entity mention in it, you are provided with some candidate entities and their information. Now, your task is to consider carefully which of the candidate entities matches the entity mention in user input.

===NOTICE===

1. The user input and entity information can be in English or Chinese.
2. If all the candidate entities are irrelevant to the input entity mention, reply [None]. However, you should not reply [None] simply because the provided entity information do not directly answer the question. If the entity mention clearly matches a candidate entity, you should definitely return it.
3. If multiple candidates are possible, just choose one that you think is most possible.
4. When the user input is an assumption or question, do not think entities are irrelevant to the input simply because their information cannot cover the assumption or question. Use your imagination how the assumption or question can be related with the candidate entities.
5. [IMPORTANT] Always remember that your task is to select the correct entity instead of answering the questions. Never try to directly answer user input in any form. Always reply [ENT 1], [ENT 2] ... [ENT n] or [NONE].
6. Try your best to ensure the entity you choose is equivalent to the input target entity. They should belong to the same type. For example, if the input target entity is William "William Shakespeare", you shouldn't choose "William Shakespeare's plays" as your answer, since the former is a person while the later are works.

===INPUT FORMAT===

You are provided with the [USER INPUT], the [TARGET ENTITY] of the target entity mention, and several candidate entities like [ENT 1], [ENT 2] ... [ENT n].

===OUTPUT FORMAT===

In order to find the correct entity, you should think step by step, and output in json format. First, you should generate your 'thought' considering the user input, the target entity mention, and all the candidate entities. Do not directly answer user query in your thought. Then, output your 'choice', which is the entity that matches the entity mention like [ENT 1], [ENT 2] ... [ENT n], or [NONE] if there is none.

===EXAMPLES===

1.

Input:

[USER INPUT]: Which Nobel laureate in Literature is best known for 'Blowing in the Wind'?

[TARGET ENTITY]: Blowing in the Wind

[ENT 1]: Blowing in the Wind (Q4928603): album by Lou Donaldson.

[ENT 2]: Blowing in the Wind (Q15921392): television series.

[ENT 3]: Blowing in the wind (Q59044759): scientific article published in Nature.

Output:

```
{
  "thought": "None of the album, tv series and scientific article seems related with Nobel Prize in Literature.",
  "choice": "[None]"
}
```

2.

Input:

[USER INPUT]: Please introduce the academic achievements of Liang Jiaqing.

[TARGET ENTITY]: Liang Jiaqing

[ENT 1]: Liang Jiaqing: Liang Jiaqing, also known as Lu Yuan, is a member of the Chinese Communist Party. He was born after the 1960s and has a university education. He is a specially appointed writer for "Chinese Writers" magazine and "Chinese Reportage Literature" magazine. Attributes: Category -> Cultural figure, Author -> The Loyal Life of a Criminal Police Captain.

[ENT 2]: Liang Jiaqing (Scholar at Fudan University): Liang Jiaqing graduated from the School of Computer Science at Fudan University and holds a Ph.D. degree. He is a well-known scholar in the field of knowledge graph and natural language processing. Attributes: Graduated from -> Fudan University.

Output:

```
{
  "thought": "The user wants to know about the academic achievements of Liang Jiaqing, so here Liang Jiaqing refers to a scholar, matching with [ENT 2].",
  "choice": "[ENT 2]"
}
```

Figure 8: Prompt for entity linking. The second example is in Chinese and translated into English.

Prompt for Question Answering:

You are an helpful and knowledgeable AI assistant. The user has issued a query, and you are provided with some related knowledge. Now, you need to think step by step to answer the user input with the related knowledge.

===REQUIREMENTS===

1. You should think step by step. First, think carefully whether you can answer this query without the provided knowledge. Second, consider how to use the related knowledge to answer the query. Then, tell me whether this query can be answered with your own knowledge and the provided knowledge. If so, answer this question. However, if the query involves a command or an assumption, you should always regard it as answerable.
2. When you are thinking, you can use and cite the provided knowledge. However, when you are generating the answer, you should pretend that you came up with the knowledge yourself, so you should not say things like "according to the provided knowledge from ..." in the "answer" part.
3. The user query and provided knowledge can be in both Chinese and English. Generate your "thought" and "answer" in the same language as the input.
4. Response in json format, use double quotes.

===INPUT FORMAT===

```
{
  "query": "<the user query that you need to answer>",
  "knowledge": "<the background knowledge that you are provided with>"
}
```

===OUTPUT FORMAT===

```
{
  "thought": "<Your thought here. Think step by step as is required.>",
  "answerable": "<yes or no. Whether you can answer this question with your knowledge and the provided knowledge. If the query involves a command or an assumption, say 'yes'.>",
  "answer": "<Your answer here, if the query is answerable.>"
}
```

===EXAMPLES===

Input:{

"query": "What is the motto of the school where Xia Mingyou graduated?",

"knowledge": "[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Xia Mingyou'], relation_aliases = ['graduated from', 'school']) ->] Xia Mingyou, school: Fudan University[find_entity_or_value(entity_aliases = ['Fudan University'], relation_aliases = ['motto']) ->] Fudan University, motto: Rich in Knowledge and Tenacious of Purpose; Inquiring with Earnestness and Reflecting with Self-practice"

}

Output:{

"thought": "Based on the background knowledge from CNDBPedia, Xia Mingyou graduated from Fudan University, and the motto of Fudan University is ' Rich in Knowledge and Tenacious of Purpose; Inquiring with Earnestness and Reflecting with Self-practice '. So the answer is ' Rich in Knowledge and Tenacious of Purpose; Inquiring with Earnestness and Reflecting with Self-practice '. This question can be answered based on the provided knowledge.",

"answerable": "yes",

"answer": " Rich in Knowledge and Tenacious of Purpose; Inquiring with Earnestness and Reflecting with Self-practice "

}

'What is Liang Jiaqing's weapon?'

Input:{

"query": "What is Liang Jiaqing's weapon?",

"knowledge": "[FROM CNDBPEDIA] Liang Jiaqing: Liang Jiaqing, also known as Lu Yuan. A member of the Chinese Communist Party, born after the 1960s, with a university education. Specially appointed writer for 'Chinese Writers' magazine and 'Chinese Reportage Literature' magazine. Attributes: Author -> The Loyal Life of a Criminal Police Captain."

}

Output:{

"thought": "According to the knowledge provided by CNDBPedia, Liang Jiaqing is an author. The provided knowledge does not mention anything about Liang Jiaqing's weapon, and authors generally do not have weapons. The question cannot be answered based on the provided knowledge or my knowledge.",

"answerable": "no"

}

Figure 9: Prompt for question answering. The examples are in Chinese and translated into English.

Prompt for Knowledge Extraction:

You are an awesome information extraction agent and your task is to extract important pieces of information (knowledge) in a structured form from unstructured text, which will be stored to build a knowledge base memory.

You should extract the following three types of knowledge:

1. Entity description:

Format: {"<entity>": "<entity description in the encyclopedic style>"}

You should extract entity description, which is the most critical information about an entity, as concise as possible, and in the encyclopedic style. For example, {"Elon Reeve Musk": "Elon Reeve Musk (born June 28, 1971) is a business magnate and investor..."}

2. Relational triple:

Format: [{"<subject>", "<predict>", "<object>"}]

You should extract relational triples (a.k.a factual triples), a common type of knowledge widely used to build knowledge graphs. The <subject> must be an entity in natural language, e.g. "United States". The <predict> is an attribute or a relationship, e.g. "population" or "located in". Attributes mainly refer to the possible attributes, features, characteristics, and parameters of an entity. Relationships connect two entities and characterize their relationships. The <object> is an attribute value or another entity, such as "over 333 million" in ["United States", "population", "over 333 million"], or "North America" in ["United States", "located in", "North America"], respectively.

3. Entity-Aspect-Content:

Format: [{"<entity>", "<aspect>", "<content>", "<question>"}]

You should also extract textual information that describes an entity in a certain aspect. This differs from relational triple in that the <content> is a longer piece of text of an entity instead of a concise value or entity. It also differs from entity description in that the <content> describes an entity in a certain aspect. Besides, generate a <question> about the <aspect> of the <entity>. For example, you should extract ["Elon Reeve Musk", "biography", "Musk was born in Pretoria, South Africa, and briefly attended the University of Pretoria before moving to Canada at age 18, acquiring citizenship through his Canadian-born mother."] to describe "Elon Reeve Musk" in terms of his "biography". The generated question can be "What is Elon Reeve Musk's biography?".

===REQUIREMENTS===

1. [IMPORTANT] Always remember that your task is to extract information from a given corpus. Never try to answer a query or a command. More importantly, only extract information from the given text, and never generate things not mentioned in the text.
2. There can be overlapped information among the three types of extracted knowledge. The same piece of information can be appropriately organized into two or more formats. For example, the corpus "Elon Reeve Musk is born on June 28, 1971." can be organized into an entity encyclopedic description like {"Elon Reeve Musk": "Elon Reeve Musk is born on June 28, 1971."}, and meanwhile, the corpus can also be organized into a relational triple like [{"Elon Reeve Musk", "born on", "June 28, 1971"}].
3. The extracted entity description should be the most critical information about the entity, in the encyclopedic style.
4. When extracting knowledge in the relational triple format of [{"<subject>", "<predict>", "<object>"}], make sure the <predict> is a clear attribute or relation. Avoid vague and broad predicts like "information", "related to". Instead, put the information into the entity description format if it's a piece of encyclopedia-like information about the entity, or put the information into the entity-aspect-content format if you can summarize a specific aspect for this piece of information.
5. The input can be in both English and Chinese. If most of the input is written in English, make sure the output is in English. If most of the input is written in Chinese, make sure the output is in Chinese.
7. Think step by step. Firstly, you need to understand what the given corpus is mainly about and then decide on the core entities you plan to extract from this corpus. Next, iterate through each core entity, coming up with a "thought" about how to extract information about this core entity and according to the generated "thought", extracting "entity_description", "relational_triple", "entity_aspect_content" for each entity in turn. Specifically, the "thought" field should give details on how to find all relevant information about the core entity from the corpus, and how to choose the appropriate formats to organize these pieces of information according to their characteristics. Make sure your "thought" covers all the relevant entities mentioned in the input. Make "thought" in the same language as the input corpus.
8. The entity names should be concise, yet accurately refer to the entity.
9. Extract as many relational triples as possible from input text.
10. Response in json format.

===OUTPUT FORMAT===

Format description: The output format is a nested json object. The top-level json object has two fields named "thought" and "knowledge" respectively. The content of the "thought" is a string; the content of the "knowledge" is another json object whose keys are core entities. You can extract one or more core entities from the corpus, e.g. <core entity 1>, <core entity 2>, etc. The value of the <core entity 1> is another json object, whose keys are "entity_description", "relational_triple", "entity_aspect_content". The value of "entity_description" is a piece of encyclopedic text. The value of "relational_triple" is a list of relational triples whose <subject> or <object> is the <core entity 1>. The value of "entity_aspect_content" is a list of [{"<entity>", "<aspect>", "<content>", "<question>"}] quaternions whose <entity> is the <core entity 1>.

Format:

```
{
  "thought": "<Your thought here. Determine the language of the input text and output in that language. Understand what the given corpus is mainly about and then decide on the core entities you plan to extract from this corpus. For each core entity, find all relevant information about the core entity and choose the appropriate formats to organize these pieces of information according to their characteristics.>",
  "knowledge": {
    "<core entity 1>": { "entity_description": "< entity description in the encyclopedic style>", "relational_triple": [{"<core entity 1>", "<predict 1>", "<object 1>"}, [{"<subject 2>", "<predict 2>", "<core entity 1>"}], "entity_aspect_content": [{"<core entity 1>", "<aspect 1>", "<long text 1>", "<question 1>"}, [{"<core entity 1>", "<aspect 2>", "<long text 2>", "<question 2>"}], ...}],
    "<core entity 2>": { "entity_description": "< entity description in the encyclopedic style>", "relational_triple": [{"<core entity 2>", "<predict 1>", "<object 1>"}, [{"<subject 2>", "<predict 2>", "<core entity 2>"}], "entity_aspect_content": [{"<core entity 2>", "<aspect 1>", "<long text 1>", "<question 1>"}, [{"<core entity 2>", "<aspect 2>", "<long text 2>", "<question 2>"}], ...}],
  }
}
```

===EXAMPLES===

1.

Input: "The Orenda OE600 is a 600 hp-class liquid-cooled 8-cylinder V-block"

Output:

```
{
  "thought": "The text is written in English, so I output it in English. The text is about the Orenda OE600, an aircraft engine designed by ...",
  "knowledge": {
    "Orenda OE600": {
      "entity_description": "The Orenda OE600 is a 600 hp-class liquid-cooled 8-cylinder V-block aircraft engine intended to re-introduce ...",
      "relational_triple": [{"Orenda OE600", "type", "600 hp-class liquid-cooled 8-cylinder V-block aircraft engine"}, ["Orenda OE600", "intended for", "aircraft normally powered by the Pratt & Whitney Canada PT6 turboprop"], ["Orenda OE600", "offers", "better fuel economy"]],
      "entity_aspect_content": [{"Orenda OE600", "design purpose", "The Orenda OE600 is intended to re-introduce piston power to aircraft normally powered by the famous Pratt & Whitney Canada PT6 turboprop."}, "What is the design purpose of Orenda OE600?", ["Orenda OE600", "features", "The Orenda OE600 is a 600 hp-class liquid-cooled 8-cylinder V-block aircraft engine that offers much better fuel economy.", "What are the features of Orenda OE600?" ... ]},
      "Orenda Aerospace": {...}
    }
  }
}
```

2. ...

Figure 10: Prompt for knowledge extraction. Certain contents about entity descriptions and in-context examples are omitted.

Input: What is the registered capital of Dong Wu Securities?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	def search(): messages = "" capital, msg = find_entity_or_value(entity_aliases = ['Dongwu Securities'], relation_aliases = ['Registered Capital', 'Capital']) messages += msg return messages	[FROM CNDBPedia][find_entity_orentity_aliases = ['Dongwu Securities'], relation_aliases = ['Registered Capital', 'Capital']) ->] Dongwu Securities, Registered Capital: 1.5 billion Yuan	W/ KB 1.5 billion Yuan. W/o KB Sorry, I am unable to provide real-time specific data, we suggest you visit the official website of Dong Wu Securities directly or contact their customer service for the most accurate information.
ChatGPT	def search(): messages = "" capital, msg = find_entity_or_value(entity_aliases = ['Dongwu Securities'], relation_aliases = ['Registered Capital', 'Registered Capital']) messages += msg return messages	[FROM CNDBPedia][find_entity_orentity_aliases = ['Dongwu Securities'], relation_aliases = ['Registered Capital', 'Registered Capital']) ->] Dongwu Securities, Registered Capital: 1.5 billion Yuan	W/ KB 1.5 billion Yuan. W/o KB The registered capital of Soochow Securities is RMB 10 billion.
Input: Which historical hero is Saber?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	def search(): messages = "" historical_figure, msg = find_entity_orentity_aliases = ['Saber'], relation_aliases = ['historical character', 'historical archetype']) messages += msg return messages	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Saber'], relation_aliases = ['Historical Character', 'Historical Archetype']) ->] Artoria Pendragon (one of the female protagonists in the Japanese text-based adventure game "fate/stay night"), Born: Ancient Britannia	W/ KB Artoria Pendragon. W/o KB Saber is a character from the Japanese anime Fate/stay night who is based on the legendary King Arthur.
ChatGPT	def search(): messages = "" hero, msg = get_entity_info(entity_aliases = ['Saber'], relation_aliases = ['Historical heroes', 'Historical figures']) messages += msg return messages	None	W/ KB None W/o KB Saber is a character from the historical legend of King Arthur. In the legend of King Arthur, Saber is a brave and upright knight who possesses a legendary sword known as Excalibur. Saber has been featured in many works of literature, movies, and games, becoming a much loved and admired heroic figure.
Input: Please write a resume for Prof. Sun Maosong that has a clear format and layout.			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	def search(): messages = "" info, msg = get_entity_info(entity_aliases = ['Sun Maosong', 'Professor Sun Maosong']) messages += msg return messages	[FROM CNDBPedia][get_entity_info(entity_aliases = ['Sun Maosong', 'Prof. Sun Maosong']) ->] Sun Maosong (Secretary of the Party Committee of the Department of Computer Science, Tsinghua University): Sun Maosong, Professor, Doctoral Supervisor, was the head of the Department of Computer Science and Technology of Tsinghua University... Attributes: Category->Industry Figures, People.	W/ KB Name: Sun Maosong Position: - Professor, Tsinghua University - Doctoral Supervisor ... Previous positions: - Chairman, Department of Computer Science and Technology, Tsinghua University. W/o KB Biography of Prof. Maosong Sun Personal Information: ... Educational Experience: 1. XX University, Ph.D., Major: XXXX, Year: XXXX-XXXX ...
ChatGPT	def search(): messages = "" info, msg = get_entity_info(entity_aliases = ['Prof. Sun Maosong']) messages += msg return messages	[FROM CNDBPedia][get_entity_info(entity_aliases = ['Prof. Sun Maosong']) ->] Sun Maosong (Secretary of the Party Committee of the Department of Computer Science, Tsinghua University): Sun Maosong, Professor, Doctoral Supervisor, was the head of the Department of Computer Science and Technology of Tsinghua University... Attributes: Category->Industry Figures, People.	W/ KB Sun Maosong Position: Professor, Doctoral Supervisor Work Experience: - Chair, Department of Computer Science and Technology, Tsinghua University ... W/o KB Name: Sun Maosong Educational Background: - Bachelor's degree: XX University, majoring in XXX, graduated in XXXX ...

Table 6: The detailed results of single-hop relational queries and a diversified instruction on CN-DBPedia.

Input: What is the main ingredient in "Ants on a Tree"? Who is the voice of Saber?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" ingredients, msg = find_entity_or_value(entity_ aliases = ['ants on a tree'], relation_aliases = ['main ingredients', 'toppings']) messages += msg voice_actor, msg = find_entity_or_value(entity_ aliases = ['Saber'], relation_aliases = ['dub', 'voice ac tor']) messages += msg return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Ants on a Tree'], relation_aliases = ['Main Ingredients', 'Ingredients']) ->] Ants on a Tree (Chinese name of the dish), Main Ingredients: Vermicelli (vermicelli), Minced Meat, Ginger, Garlic, Scallions, Onion [find_entity_or_value(entity_aliases = ['Saber'], relation_aliases = ['Dubbing', 'Voice Actor']) ->] saber (Spirit from the game "For Whom the Alchemy is Made"), Dubbed by: Ayako Kawasumi	W/ KB The main ingredient in Ants on a Tree is stir-fried vermicelli with minced pork, and Saber is voiced by Ayako Kawasumi. W/o KB The main ingredient of "Ants on a Tree" is stir-fried vermicelli with minced pork, and Saber is voiced by Ayako Kawasumi.
ChatGPT	<pre>def search(): messages = "" main_ingredient, msg = find_entity_or_value(entity_ aliases = ['ants on a tree'], relation_aliases = ['main ingredient', 'main ingredient', 'main ingredient']) messages += msg voice_actor, msg = find_entity_or_value(entity_ aliases = ['Saber'], relation_aliases = ['who the dub is', 'who the voice is', 'voice actor']) messages += msg return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Ants on a tree'], relation_aliases = ['Main Ingredients', 'Main Ingredients', 'Main Ingredients']) ->] Ants on a tree (Chinese dish name), Main Ingredients: Vermicelli (vermicelli), minced meat, ginger, garlic, scallions	W/ KB No enough knowledge to answer. W/o KB Ants on the Tree is a traditional Sichuan dish whose main ingredients are minced pork and vermicelli... Regarding the voice of Saber, Saber is a character in the Fate/stay night series, and the voice actors for Saber vary depending on the production and version. Some of the more well-known voice actors are Ayako Kawasumi and Maya Sakamoto.
Input: What are the titles of the poet writing Quiet Night Thoughts (Jing Ye Si) ?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" author, msg = find_entity_or_value(entity_aliases = ['Quiet Night Thoughts'], relation_aliases = ['author', 'creator', 'writer']) messages += msg titles, msg = find_entity_or_value(entity_aliases = author, relation_aliases = ['title', 'also known as', 'ap pellation']) messages += msg return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases=['Quiet Night Thoughts'], relation_aliases=['author', 'creator', 'writer'])->]"Quiet Night Thoughts" (A poem by Li Bai), Author: Li Bai, The one who orders the destiny. [find_entity_or_value(entity_aliases=['Li Bai', The one who orders the destiny], relation_aliases=['title', 'also known as', 'appellation'])->] Li Bai (Famous poet of the Tang Dynasty), also known as: Qinglian Jushi, Zhixianren.	W/ KB The poet of "Quiet Night Thoughts" has two titles: "Qinglian Jushi" and "Zhixianren". W/o KB The poet of "Quiet Night Thoughts" is Li Bai from the Tang Dynasty. He has several titles, including "Poetry Immortal", "Wandering Poet", and "Wine Immortal". His poetic talent is outstanding, and his works are widely circulated, revered by later generations as the "God of Poetry".
ChatGPT	<pre>def search(): messages = "" poet, msg = find_entity_or_value(entity_aliases = ['Quiet Night Thoughts'], relation_aliases = ['author of "Quiet Night Thoughts"', 'creator of "Quiet Night Thoughts"']) messages += msg titles, msg = find_entity_or_value(entity_aliases = [poet], relation_aliases = ['title']) messages += msg return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Quiet Night Thoughts'], relation_aliases = ['author of "Quiet Night Thoughts"', 'creator of "Quiet Night Thoughts"']) ->] "Quiet Night Thoughts" (A poem by Li Bai), Author: Li Bai, The one who orders the destiny.	W/ KB Li Bai. W/o KB "Quiet Night Thoughts" is a work written by Tang Dynasty poet Li Bai. Li Bai is referred to as the "Poetry Immortal" and "Poetry Sage", and he is also one of the most influential poets in Chinese literary history.
Input: Do you know when Albert II's father was born?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" father, msg = find_entity_or_value(entity_aliases = ['Albert II'], relation_aliases = ['father', 'father is', 'dad']) messages += msg if father: birth_date, msg = find_entity_or_value(entity_ aliases = father, relation_aliases = ['birth_date', 'born on']) messages += msg return messages</pre>	[find_entity_or_value(entity_aliases = ['Albert II'], relation_aliases = ['Father', 'Father was', 'Dad']) ->] Albert II, Father: Rainier III [find_entity_or_value(entity_aliases = ['Albert II'], relation_aliases = ['Father', 'Dad']) ->] Albert II, Father: Rainier III [find_entity_or_value(entity_aliases = ['Rainier III'], relation_aliases = ['Date of Birth', 'Born']) ->] Rainier III, Date of Birth: May 31, 1923	W/ KB May 31, 1923 . W/o KB Albert II's father was Albert I, who was born on April 8, 1875.
ChatGPT	<pre>def search(): messages = "" father, msg = find_entity_or_value(entity_aliases = ['Albert II'], relation_aliases = ['who is the father', 'fa ther of ']) messages += msg if father: birth_date, msg = find_entity_or_value(entity_ aliases = father, relation_aliases = ['date of birth']) messages += msg return messages</pre>	None	W/ KB None W/o KB Albert II's father was Albert I, who was born on August 26, 1819.

Table 7: The detailed results of mixed queries and multi-hop relational queries on CN-DBPedia.

Input: Who were the winners of the first Nobel Prize? What prizes did they each receive?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" first_winners, msg = find_entity_or_value(entity_aliases = ['Nobel Prize'], relation_aliases = ['first winner', 'first recipient']) messages += msg for winner in first_winners: award, msg = find_entity_or_value(entity_aliases = [winner], relation_aliases = ['awarded', 'award']) messages += msg return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Nobel Prize'], relation_aliases = ['first winner', 'first recipient']) ->] Nobel Prize, first winner: Wilhelm Conrad Röntgen, Jacobus Henricus van 't Hoff, Emil Adolf von Behring, Sully Prudhomme, Henry Dunant, Frédéric Passy.[find_entity_or_value(entity_aliases = ['Wilhelm Conrad Röntgen'], relation_aliases = ['awarded', 'award']) ->] Wilhelm Conrad Röntgen, Occupation: Physicist. ... [find_entity_or_value(entity_aliases = ['Frédéric Passy'], relation_aliases = ['awarded', 'award']) ->] Frédéric Passy, Major Achievement: First recipient of the Nobel Peace Prize.	W/ KB The inaugural winners of the Nobel Prize included Wilhelm Conrad Röntgen, Jacobus Henricus van 't Hoff, Emil Adolf von Behring, Sully Prudhomme, Henry Dunant, and Frédéric Passy. They respectively received the Nobel Prizes in Physics, Chemistry, Medicine, Literature, and Peace. W/o KB The inaugural winners of the Nobel Prize were: 1. Nobel Prize in Physics: Wilhelm Conrad Röntgen, who ... 2. Nobel Prize in Chemistry: Jacobus Henricus van 't Hoff, who ... 3. Nobel Prize in Physiology or Medicine: Emil Adolf von Behring, who ... 4. Nobel Prize in Literature: Sully Prudhomme, who ... 5. Nobel Peace Prize: Henry Dunant, who ... Frédéric Passy, who ...
ChatGPT	<pre>def search(): messages = "" first_winners, msg = find_entity_or_value(entity_aliases = ['Nobel Prize'], relation_aliases = ['first winner', 'first recipient']) messages += msg awards = [] for winner in first_winners: award, msg = find_entity_or_value(entity_aliases = [winner], relation_aliases = ['award']) messages += msg awards.append(award[0]) messages += f'first winner is {first_winners}, they respectively received {awards}.' return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Nobel Prize'], relation_aliases = ['first winner', 'first recipient']) ->] Nobel Prize, first winner: Wilhelm Conrad Röntgen, Jacobus Henricus van 't Hoff, Emil Adolf von Behring, Sully Prudhomme, Henry Dunant, Frédéric Passy.[find_entity_or_value(entity_aliases = ['Wilhelm Conrad Röntgen'], relation_aliases = ['award']) ->] Wilhelm Conrad Röntgen, Occupation: Physicist. ... (Did not retrieve results for all the laureates.)	W/ KB Wilhelm Conrad Röntgen received the Physics Prize, Jacobus Henricus van 't Hoff received the Chemistry Prize, and Emil Adolf von Behring received the Medicine Prize. (Miss three winners.) W/o KB The first Nobel Prizes were awarded in 1901. Here are the inaugural Nobel Prize winners and their respective categories: 1. Physics Prize: Wilhelm Conrad Röntgen and Wilhelm Hertz 2. Chemistry Prize: Jacobus Hansen van 't Hoff and Hermann Emil Fischer 3. Physiology or Medicine Prize: Emil von Behring and William Ramsay 4. Literature Prize: José Echegaray and Selma Lagerlöf 5. Peace Prize: Henry Dunant and Frédéric Passy ...
Input: Is the lyricist, composer, and arranger of <i>Li Bai</i> the same person?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" lyricist, msg = find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['lyrics writing', 'lyricist']) messages += msg composer, msg = find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['compose', 'composer']) messages += msg arranger, msg = find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['arrange', 'arranger']) messages += msg if lyricist == composer == arranger: messages += 'The lyricist, composer, and arranger of "Li Bai" are the same person.' else: messages += 'The lyricist, composer, and arranger of "Li Bai" are not the same person.' return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['lyrics writing', 'lyricist']) ->] Li Bai (a song sung by Li Ronghao), lyricist: Li Ronghao[find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['compose', 'composer']) ->] Li Bai (a song sung by Li Ronghao), arrange: Li Ronghao [find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['arrange', 'arranger']) ->] Li Bai (a song sung by Li Ronghao), arrange: Li Ronghao. The song <i>Li Bai</i> has its lyrics, composition, and arrangement all done by the same person.	W/ KB Yes, the song <i>Li Bai</i> has its lyrics, composition, and arrangement all done by the same person, Li Ronghao. W/o KB The lyricist, composer, and arranger of the song <i>Li Bai</i> are not the same person. The lyrics and composition of this song were done by Ashin (A Xin/A Pa), while the arrangement was done jointly by Mayday and Goro Liu.
ChatGPT	<pre>def search(): messages = "" lyricist, msg = find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['lyrics writing']) messages += msg composer, msg = find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['compose']) messages += msg arranger, msg = find_entity_or_value(entity_aliases = ['Li Bai'], relation_aliases = ['arrange']) messages += msg if len(lyricist) == 1 and len(composer) == 1 and len(arranger) == 1 and lyricist[0] == composer[0] == arranger[0]: messages += 'Is the same person' else: messages += 'Not the same person' return messages</pre>	None	W/ KB None W/o KB The lyrics, composition, and arrangement of <i>Li Bai</i> were all done by the same person, the composer Li Ronghao.

Table 8: The detailed results of multi-hop relational queries on CN-DBPedia.

Input: What is the relationship between Ronald Lee and Li Bai?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" relationship, msg = find_relationship(entity1_aliases = ['Li Ronghao'], entity2_aliases = ['Li Bai']) messages += msg return messages</pre>	[FROM CNDBPedia][find_relationship(entity1_aliases = ['Li Ronghao'], entity2_aliases = ['Li Bai']) ->] Li Ronghao, Representative Work: Li Bai	W/ KB Li Ronghao created a song titled 'Li Bai' which is about his relationship with Li Bai. W/o KB Li Ronghao and Li Bai are not directly related. Li Ronghao is a famous singer and music producer in modern times...
ChatGPT	<pre>def search(): messages = "" relationship, msg = find_relationship(entity1_aliases = ['Li Ronghao'], entity2_aliases = ['Li Bai']) messages += msg return messages</pre>	[FROM CNDBPedia][find_relationship(entity1_aliases = ['Li Ronghao'], entity2_aliases = ['Li Bai']) ->] Li Ronghao, Representative Work: Li Bai	W/ KB Li Ronghao and Li Bai are the same person. W/o KB Li Ronghao and Li Bai are two different people with no direct relationship. Li Ronghao is a famous singer and music producer in modern China, while Li Bai is ...
Input: Who is taller, Yao Ming or Saber?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" yao_height, msg = find_entity_or_value(entity_ aliases = ['Yao Ming'], relation_aliases = ['height']) messages += msg saber_height, msg = find_entity_or_value(entity_ aliases = ['Saber'], relation_aliases = ['height']) messages += msg if yao_height[0] > saber_height[0]: messages += 'Yao Ming is taller than Saber.' elif yao_height[0] < saber_height[0]: messages += 'Saber is taller than Yao Ming' else: messages += 'Yao Ming and Saber are of the same height.' return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Yao Ming'], relation_aliases = ['height'], 'altitude')] ->] Yao Ming (Chairman of the Asian Basketball Association, Chairman of the Chinese Basketball Association), height: 226 cm [find_entity_or_value(entity_aliases = ['Saber'], relation_aliases = ['height', 'altitude']) ->] saber , height: 154 cm Yao Ming is taller than Saber."	W/ KB Yao Ming is taller than Saber. W/o KB Yao Ming's height is 2.26 meters, and Saber (a fictional character from the Fate series) is 1.54 meters tall. So, Yao Ming is taller than Saber.
ChatGPT	<pre>def search(): messages = "" yao_height, msg = find_entity_or_value(entity_ aliases = ['Yao Ming'], relation_aliases = ['height']) messages += msg saber_height, msg = find_entity_or_value(entity_ aliases = ['Saber'], relation_aliases = ['height']) messages += msg return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Yao Ming'], relation_aliases = ['height']) ->] Yao Ming (Chairman of the Asian Basketball Association, Chairman of the Chinese Basketball Association), height: 226 cm	W/ KB Yao Ming is taller than Saber. W/o KB Yao Ming and Saber are figures from completely different fields, making it difficult to directly compare their heights. Yao Ming is a famous basketball player from China, standing at 2.26 meters, while Saber is a character from the Japanese animation "Fate/stay night", a fictional character. Therefore, in real life, Yao Ming is taller than Saber.
Input: Which one has a larger registered capital, Dongwu Securities or Xingye Securities?			
Model	Search Code	Retrieved Knowledge	Answer
GPT-4	<pre>def search(): messages = "" ew_capital, msg = find_entity_or_value(entity_aliases = ['Dongwu Securities'], relation_aliases = ['registered capital', 'capital']) messages += msg xy_capital, msg = find_entity_or_value(entity_aliases = ['Xingye Securities'], relation_aliases = ['registered capital', 'capital']) messages += msg if ew_capital and xy_capital: if ew_capital[0] > xy_capital[0]: messages += 'Dongwu Securities has a larger registered capital.' elif ew_capital[0] < xy_capital[0]: messages += 'Xingye Securities has a larger registered capital.' else: messages += 'Dongwu Securities and Xingye Securities have the same registered capital.' return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Dongwu Securities'], relation_aliases = ['registered capital', 'capital']) ->] Dongwu Securities, registered capital: 1.5 billion yuan[find_entity_or_value(entity_aliases = ['Xingye Securities'], relation_aliases = ['registered capital', 'capital']) ->] Xingye Securities, registered capital: 6.697 billion yuan (2018)Xingye Securities has a larger registered capital.	W/ KB Xingye Securities has a larger registered capital. W/o KB I'm sorry, I can't provide real-time company registered capital information, as these data may change over time. It is recommended that you directly visit the official websites of these two companies or contact their customer service to get the most accurate information.
ChatGPT	<pre>def search(): messages = "" eastmoney_capital, msg = find_entity_or_value(entity_aliases = ['Dongwu Securities'], relation_aliases = ['registered capital']) messages += msg xingye_capital, msg = find_entity_or_value(entity_ aliases = ['Xingye Securities'], relation_aliases = ['registered capital']) messages += msg return messages</pre>	[FROM CNDBPedia][find_entity_or_value(entity_aliases = ['Dongwu Securities'], relation_aliases = ['registered capital']) ->] Dongwu Securities, registered capital: 1.5 billion yuan	W/ KB (Error when parsing LLM response into json format.). W/o KB Dongwu Securities has a larger registered capital. According to public data, the registered capital of Dongwu Securities is RMB 15 billion, while the registered capital of Xingye Securities is RMB 10 billion.

Table 9: The detailed results of comparison queries on CN-DBPedia.